

1. HTML5 Form Controls & Input Types

HTML FORM

```
<!-- Inputs: Text, Number, Password -->
<input type="text" id="uName" name="user"
placeholder="Name">
<input type="number" id="valln" name="qty"
min="0" step="any">
<input type="password" id="pwdln" name="pass">
<!-- Dropdown & Textarea -->
<select id="catSel" name="cat"><option
value="CSE">CSE</option><option
value="EEE">EEE</option></select>
<textarea id="txtArea" name="desc" rows="1"
placeholder="Note"></textarea>
<!-- Radios, Checkboxes & Action Button -->
<input type="radio" name="lvl" value="UG"
checked> UG
<input type="checkbox" name="sk" value="JS"> JS
<button type="button" id="btn"
onclick="runApp()">Submit</button> <span
id="output"></span>
```

2. DOM Extraction, Parsing & Controls

DOM CORE

```
// 1. Reading & Strict Conversions
let str =
document.getElementById("uName").value;
let intN =
parseInt(document.getElementById("valln").value,
10);
let fltN =
parseFloat(document.getElementById("valln").valu
e);

// 2. Radio & Checked Checkbox Array
let rad =
document.querySelector('input[name="lvl"]:checked')
?.value;
let chk =
Array.from(document.querySelectorAll('input[name
="sk"]:checked'))
.map(c => c.value);

// 3. Writing Output & Disabling
let out = document.getElementById("output");
out.innerHTML = `<b>Score:</b> ${fltN}`;
out.style.color = (fltN >= 50) ? "green" :
"red";
document.getElementById("valln").disabled =
true; // lock
document.getElementById("btn").disabled = true;
// lock
```

Trap: .value is ALWAYS a string! "20"+5="205". Always parse with parseFloat() or parseInt()!

3. Math API & Universal Random Formula

MATH ENGINE

```
// Random integer: min to max (both inclusive)
let rnd = Math.floor(Math.random() * (max -
min + 1)) + min;

Range    Exact Code Snippet
500 .. 5000    Math.floor(Math.random() * 4501) + 500
200 .. 400     Math.floor(Math.random() * 201) + 200
1 .. 100       Math.floor(Math.random() * 100) + 1

Math.abs(x - 80); // absolute: |-5| = 5
Math.round(x * 100)/100; // 2 decimal: 90.67
x.toFixed(2); // string "90.67"
```

4. Dynamic HTML Table Row Builder

DOM INJECT

```
// Append row to existing <tbody id="tblBody">
document.getElementById("tblBody").innerHTML +=
<tr><td>${str}</td><td>${fltN}</td></tr>;
```

5. Basic Clean CSS (Form, Table, Input)

CSS FORMAT

```
/* Form & Inputs (Clean & Centered) */
form { width: 300px; margin: 4px auto; padding:
6px; border: 1px solid #cbd5e1; }
label { display: block; margin-top: 3px; font-
weight: bold; }
input[type="text"], input[type="number"], select {
width: 100%; padding: 3px; margin-top: 2px;
border: 1px solid #94a3b8; }
button, input[type="submit"] {
margin-top: 4px; padding: 3px 10px;
background: #0284c7; color: #fff;
border: none; cursor: pointer; border-radius:
2px; }

/* Formatted Table (Collapse + Zebra) */
table { width: 100%; border-collapse: collapse;
margin-top: 4px; }
th, td { border: 1px solid #cbd5e1; padding: 2px
4px; text-align: left; }
th { background: #f1f5f9; font-weight: bold; }
tr:nth-child(even) { background: #f8fafc; } /*
Zebra */

/* Formatting & State Classes */
.error { color: #b91c1c; font-weight: bold; }
.success { color: #16a34a; }
.center { text-align: center; }
```

6. Universal JS 5-Stage App Pipeline

Q1 MASTER

```
<input type="text" id="fName"
placeholder="Subject/Item">
<input type="number" id="fVal"
placeholder="Hours/Score">
<button id="btn"
onclick="processApp()">Submit</button>
<p id="msgOut"></p>
<script>
// STAGE 0: State Variables OUTSIDE Function!
let runningTotal = 0, sessionCount = 0;
let itemHistory = [];

function processApp() {
// STAGE 1: Extract & Parse
let name =
document.getElementById("fName").value;
let val =
parseFloat(document.getElementById("fVal").value
);
let out =
document.getElementById("msgOut");

// STAGE 2: Mutate State & History
runningTotal += val; sessionCount++;
itemHistory.push(name);

// STAGE 3: Compute Statistical Averages
let avg = runningTotal / sessionCount;

// STAGE 4: Evaluation Ladder (Highest First!)
let feedback = "";
if (runningTotal >= 100) feedback =
"Goal Achieved!";
else if (runningTotal >= 60) feedback =
"Good Progress!";
else feedback =
"Keep Working!";

// Guard: Alert on excessive attempts
if (sessionCount > 7 && runningTotal < 100)
{
feedback += " (High Attempts Warning!)";
}

// STAGE 5: Render DOM Feedback
out.innerHTML = `Total: ${runningTotal} | Avg:
${avg.toFixed(2)} | ` +
`History: ${itemHistory.join(", ")} | Status:
<b>${feedback}</b>`;

// STAGE 6: Termination Lock
if (runningTotal >= 100 || sessionCount >=
10) {
document.getElementById("fVal").disabled =
true;
document.getElementById("btn").disabled =
true;
}
}</script>
```

7. Specialized JS Logic Engines

ALGORITHMS

```
// Engine A: Target Comparator & Difference
let diff = Math.abs(actual - target);
if (actual === target) msg = "Target met exactly
(0 difference)";
else if (actual > target) msg = `Above target by
${diff}`;
else msg = `Below target by
${diff}`;

// Engine B: Secret Guess Machine (5 Attempts)
if (guess === secret) {
feed = "Correct!"; input.disabled = true;
} else if (attempts >= 5) {
feed = "Out of guesses!"; input.disabled =
true;
} else if (guess > secret) { feed = "Too high!"; }
else { feed = "Too low!"; }

// Engine C: Rule Scoring & Regex Evaluator
let score = 0;
if (p.length >= 6) score +=
Math.floor((p.length - 6)/2) * 10;
if (/^[A-Z]/.test(p)) score += 15; // Upper
if (/^[a-z]/.test(p)) score += 15; // Lower
if (/^[0-9]/.test(p)) score += 20; // Digit
if (/^[!@#%$*&#]/.test(p)) score += 25; // Symbol

// Engine D: Dual-Metric Health Risk Formula
let risk = (100 - spo2) * 2 + Math.abs(hr - 80)
* 0.5;
let tier = (risk <= 10) ? "SAFE" : (risk <= 20 ?
"WARNING" : "DANGER");
```

8. JS Array, Object & Filter Toolkit

UTILITY

```
// Array Operations
let arr = [10, 20, 30];
arr.push(40); // append element
arr.pop(); // remove last element
arr.join(","); // "10, 20, 30"
arr.slice(0, 2); // copy sub-array without
mutating
arr.sort((a, b) => a - b); // ascending numeric sort
arr.sort((a, b) => b - a); // descending numeric
sort

// String Splitting & Cleanups
"English-3".split("-"); // ["English", "3"] (subject + hr)
str.trim(); // remove whitespace
str.toLowerCase(); // case-insensitive
comparison
str.includes("Beach"); // substring check

// Search in Array of Objects
let filtered = items.filter(it =>

it.name.toLowerCase().includes(query.toLowerCase
e()) &&
it.price <= maxP
);
```

9. Financial Discount & GPA Calculator

CALC

```
// Discount & Tax
let sub = price * qty;
let discPct = (sub >= 5000) ? 0.20 : ((sub >=
2000) ? 0.10 : 0);
let discAmt = sub * discPct;
let grandTot = (sub - discAmt) * 1.05; // 5% tax
included

// Weighted GPA Accumulator
function getGP(m) {
if (m >= 80) return 4.0; if (m >= 70) return
3.5;
if (m >= 60) return 3.0; return 0.0;
}
totPts += getGP(marks) * cr; totCr += cr;
let gpa = totPts / totCr;
```

10. PHP One-File Form Processor

PHP CORE

```
<form method="post" action="">
Input A: <input type="number" name="valA"
required>
Input B: <input type="number" name="valB"
required>
<input type="submit" name="btnSubmit"
value="Calculate">
</form>
<?php
(isset($_POST["btnSubmit"])) {
$a = floatval($_POST["valA"]);
$b = floatval($_POST["valB"]);
$res = $a * $b;
echo "<p>Result: <b>". $res . "</b></p>";
}
?>
```

PHP Rules: Variables always start with \$. String concatenation is dot (.), NOT +! Button must have name="btnSubmit".

11. PHP Functions, Arrays & Sessions

PHP API

```
// Math & Array Functions
ceil(30 / 8); // 4 (round UP to whole item)
floor(30 / 8); // 3 (round DOWN)
round(3.754, 2); // 3.75
min($a, $b, $c); // lowest (use for best 2 of 3
marks)
number_format(8400); // "8,400" (comma
formatted)
array_sum($arr); // sum of array elements
asort($scores); // sort associative array by
value

// Sessions & Security
session_start();
$_SESSION["user"] = $name; // store session
htmlspecialchars($str); // XSS sanitization
```

12. Universal ceil() Allocator Engine

Q2 CORE

"Cannot order/book partial items" → Always ceil():
// 1. Universal 5-Line Math Formula
\$needed = \$demandCount * \$ratePerUnit;
\$items = ceil(\$needed / \$capacityPerItem);
// Round UP
\$supplied = \$items * \$capacityPerItem; // Total capacity
\$unused = \$supplied - \$needed;
// Leftover/empty
\$wasted = \$unused * \$costPerSingleUnit;
// Money wasted

// 2. Efficiency Ratio / Category
\$ratio = \$unused / \$supplied;
if (\$ratio <= 0.10) \$status = "Efficient";
elseif (\$ratio <= 0.25) \$status = "Acceptable";
else \$status = "Wasteful";

Distractor Rule: Fixed venue/screen rental (15,000 / 25,000 BDT) is NEVER in the wasted formula! Wasted money is strictly unused * unitPrice!

13. Progress Classifier & Best-of-3

PHP LOGIC

// Pattern A: Best 2 of 3 CT Marks (Drop lowest)
\$ctAvg = (\$ct1 + \$ct2 + \$ct3 - min(\$ct1, \$ct2, \$ct3)) / 2;
\$total = \$ctAvg + \$midterm + \$final;
\$status = (\$total > 54) ? "Passed" : "Failed";

// Pattern B: Progress Classifier & Target Delta
\$total = \$rate * \$count; // items*days OR credits*courses
if (\$total >= 500) \$tier = "Excellent"; // 261:
120
elseif (\$total >= 300) \$tier = "Good"; // 261: 90
elseif (\$total >= 150) \$tier = "Average"; // 261: 60
else \$tier = "Poor";

// Target Comparison Delta
if (\$total == \$target) {
\$msg = "Target met exactly (0 difference)";
} elseif (\$total > \$target) {
\$msg = "Above target by " . (\$total - \$target);
} else {
\$msg = "Below target by " . (\$target - \$total);
}
}

14. PHP OOP Class Architecture

OOP SPEC

```
class Product {  
    public $title;  
    public $price;  
    public $qty;  
  
    // Constructor (TWO underscores: __construct)  
    public function __construct($t, $p, $q) {  
        $this->title = $t;  
        $this->price = $p;  
        $this->qty = $q;  
    }  
  
    public function calculateTotal() {  
        return $this->price * $this->qty;  
    }  
  
    public function printDetails() {  
        echo "Title: " . $this->title . "<br>";  
        echo "Total: " . $this->calculateTotal() . "  
<br>";  
    }  
}  
  
// Object Instantiation & Calling  
$item = new Product("Web Book", 500, 3);  
$item->printDetails();  
  
OOP Rules: $this->property (NO $ after ->). Method calls use $obj->method().
```

15. PHP Date & Overdue Days Engine

DATES

// Day difference between two dates
\$due = strtotime("2026-09-10");
\$curr = strtotime("2026-09-22");
\$daysOverdue = (\$curr - \$due) / 86400; // 86400s per day
if (\$daysOverdue > 0) {
\$penalty = \$daysOverdue * 2.00; // 2 BDT per day
}

16. SQL DDL & Data Types

SCHEMA SETUP

```
CREATE DATABASE IF NOT EXISTS uiu_db;  
USE uiu_db;  
  
CREATE TABLE records (  
    id INT PRIMARY KEY AUTO INCREMENT,  
    title VARCHAR(60) NOT NULL,  
    category VARCHAR(30),  
    stock INT DEFAULT 0,  
    fee DECIMAL(10,2), -- Money: 0.00  
    rating FLOAT, -- 4.5  
    reg_date DATE, -- '2026-09-23'  
    status VARCHAR(20)  
);  
  
INSERT INTO records (title, category, stock, fee, status) VALUES  
('Web Dev', 'CSE', 10, 45.00, 'Active'),  
('Database', 'CSE', 5, 30.00, 'Overdue');  
  
Quote Escape: Double single quotes to insert apostrophe: 'Cox's Bazar'.
```

17. Deep SQL Query Vault

WEAK POINT FOCUS

Pattern 1 — Selective Filters (OR, LIKE, BETWEEN, IN):
-- OR with Substring search + Sorting DESC
SELECT title, category, rating FROM records
WHERE rating > 4.5 OR title LIKE '%Beach%'
ORDER BY rating DESC;

SELECT * FROM records WHERE rating
BETWEEN 4.0 AND 4.5;
SELECT * FROM records WHERE category IN
('CSE', 'EEE');

Pattern 2 — Group Aggregations with HAVING:
-- HAVING filters aggregated groups (WHERE cannot!)
SELECT category, COUNT(*) AS total_items,
AVG(rating) AS avg_rate
FROM records
GROUP BY category
HAVING COUNT(*) > 1 AND AVG(rating) > 4.4
ORDER BY avg_rate DESC;

Golden Rule: Every non-aggregated selected column MUST be in GROUP BY! Use HAVING for aggregate conditions, never WHERE!

Pattern 3 — Guarded Conditional UPDATE:
-- Arithmetic update with ceiling guard (prevent overshoot)
UPDATE records
SET fee = fee * 1.10
WHERE fee > 20.00 AND fee * 1.10 <= 50.00;

-- Conditional grade swap with exclusion
UPDATE records
SET status = 'Grace Period', fee = 0
WHERE status = 'Overdue' AND score < 7;

18. Subqueries, Computed Totals & CASE

ADVANCED SQL

-- 1. Grand Net Worth (Multiplication inside SUM)
SELECT SUM(fee * stock) AS grand_total
FROM records;

-- 2. Subquery: Select items above overall average
SELECT title, fee FROM records
WHERE fee > (SELECT AVG(fee) FROM records);

-- 3. Correlated Subquery CASE (Top Seller in OWN category)
SELECT title, fee,
CASE
 WHEN fee > (SELECT AVG(r2.fee) FROM records r2 WHERE r2.category = r1.category)
 THEN 'Top Seller' ELSE 'Regular Seller'
END AS seller_tier
FROM records r1;

-- 4. Date functions & Guarded DELETE
SELECT CONCAT(first_name, ' ', last_name) AS full_name,
DATEDIFF(NOW(), reg_date) AS days_registered FROM records;
DELETE FROM records WHERE status = 'Inactive' AND stock = 0;

19. Comprehensive SQL JOIN Patterns

JOINS

-- 1. INNER JOIN across two tables
SELECT s.name, c.title, e.grade
FROM student s
INNER JOIN enrollment e ON s.id = e.student_id
INNER JOIN course c ON e.course_id = c.id;

-- 2. LEFT JOIN with Aggregation (Count per dept)
SELECT d.name, COUNT(e.id) AS emp_count
FROM department d
LEFT JOIN employee e ON d.id = e.dept_id
GROUP BY d.id;

-- 3. Multi-Table Join with Computed Sum
SELECT c.name, SUM(o.qty * p.price) AS total_spent
FROM customer c
JOIN orders o ON c.id = o.customer_id
JOIN product p ON o.product_id = p.id
GROUP BY c.id;

20. Master PHP ↔ MySQL Bridge Skeleton

WRITE IN Q3

"Write full PHP-MySQL code, not just SQL" → Write this entire file!

<?php
\$conn = new mysqli("localhost", "root", "", "uiu_db");
if (\$conn->connect_error) {
die("Connection failed: " . \$conn->connect_error);
}

// SELECT Execution & Loop
\$sql1 = "SELECT category, COUNT(*) AS total_items, AVG(fee) AS avg_fee
FROM records GROUP BY category HAVING COUNT(*) > 1 ORDER BY avg_fee DESC";
\$result1 = \$conn->query(\$sql1);

if (\$result1 && \$result1->num_rows > 0) {
while (\$row = \$result1->fetch_assoc()) {
echo \$row["category"] . " : " .
\$row["total_items"] .
" (Avg: " . \$row["avg_fee"] . ")
";
} else { echo "0 results
"; }

// UPDATE Execution with Check
\$sql2 = "UPDATE records SET fee = fee * 1.10 WHERE fee > 20 AND fee * 1.10 <= 50";
if (\$conn->query(\$sql2) === TRUE) {
echo "Updated (" . \$conn->affected_rows . " rows)
";
}

\$conn->close();
?>

Column Alias Law: \$row["total_items"] works ONLY because of AS total_items in SQL! Always alias aggregated columns.

21. HTML Table Rendering from Database

HTML TABLE

```
echo "<table border='1'><tr><th>ID</th><th>Title</th><th>Fee</th></tr>";  
while ($row = $result->fetch_assoc()) {  
echo "<tr><td>{$row['id']}</td><td>{$row['title']}</td><td>{$row['fee']}</td></tr>";  
}  
echo "</table>";
```

22. Dynamic Search & Prepared SQL

SECURITY

```
$search = "%" . $_GET["keyword"] . "%";  
$stmt = $conn->prepare("SELECT * FROM records WHERE title LIKE ? AND fee > ?");  
$stmt->bind_param("sd", $search, $minFee); // s=str, d=double, i=int  
$stmt->execute();  
$res = $stmt->get_result();  
while ($r = $res->fetch_assoc()) { echo $r["title"] . "<br>"; }  
$stmt->close();
```